



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNM
TECNOLOGICO NACIONAL DE
MEXICO



TECNOLOGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA

**SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

SEMESTRE:
Enero-Agosto 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
PATRONES DE DISEÑO

TÍTULO ACTIVIDAD:
DOCUMENTACIÓN PROYECTO FINAL

UNIDAD A EVALUAR:
5

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:
MENDOZA SUAREZ IVAN GUSTAVO 22210910
FELIX SANDOVAL JOSE GUADALUPE
ORTEGA RAMOS YAHIR 21212578
SOTELO RUBIO RODRIGO 21212053

NOMBRE DEL MAESTRO (A):
MARIBEL GUERRERO LUIS



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNM
TECNOLOGICO NACIONAL DE
MEXICO



Índice

Introducción.....	3
Funcionamiento del programa.....	3
Flujo general del programa.....	4
Patrones de Diseño utilizados.....	6
Compatibilidad entre los patrones.....	7
Diagrama UML.....	8
Diagrama de estados.....	9
Conclusiones.....	9

Introducción

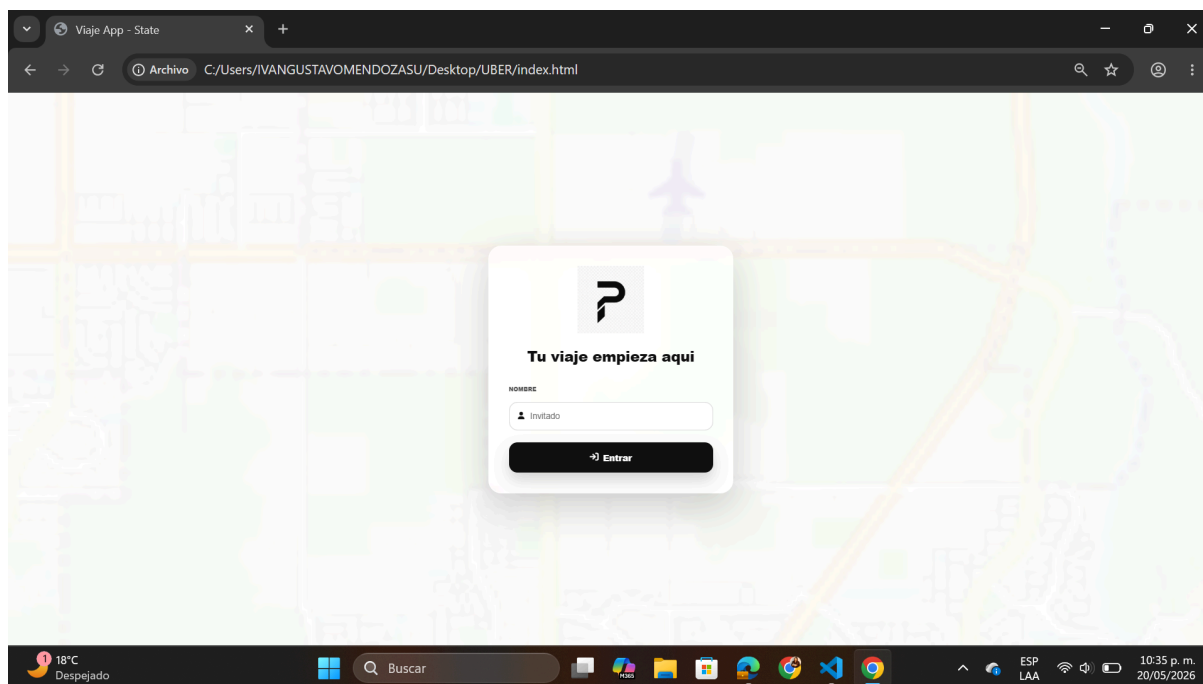
UberITT es una aplicación web inspirada en el funcionamiento básico de plataformas de transporte como Uber. El proyecto fue desarrollado utilizando tecnologías web como HTML, CSS y JavaScript, implementando distintos patrones de diseño para mejorar la organización, escalabilidad y mantenimiento del sistema.

El objetivo principal del proyecto es simular el flujo completo de un viaje dentro de una plataforma de transporte, desde la selección del origen y destino hasta la asignación del conductor, cálculo de tarifas y procesamiento del pago.

Además de ofrecer una experiencia visual e interactiva mediante mapas y animaciones, el sistema permite demostrar cómo los patrones de diseño ayudan a resolver problemas comunes en el desarrollo de software moderno.

Funcionamiento del programa

El sistema funciona como un simulador de viajes tipo Uber donde el usuario puede interactuar con diferentes elementos del sistema.



1.1 Inicio de sesion para el usuario

Flujo general del programa

1. El usuario selecciona un punto de origen.
2. Después selecciona el destino.
3. El sistema calcula la ruta y la tarifa estimada.
4. El usuario elige el tipo de viaje y el método de pago.
5. Se confirma el pedido.
6. El sistema busca y asigna un conductor disponible.
7. El conductor se dirige al punto de origen.
8. El viaje inicia.
9. El conductor se mueve hacia el destino.
10. El viaje finaliza y se procesa el pago.

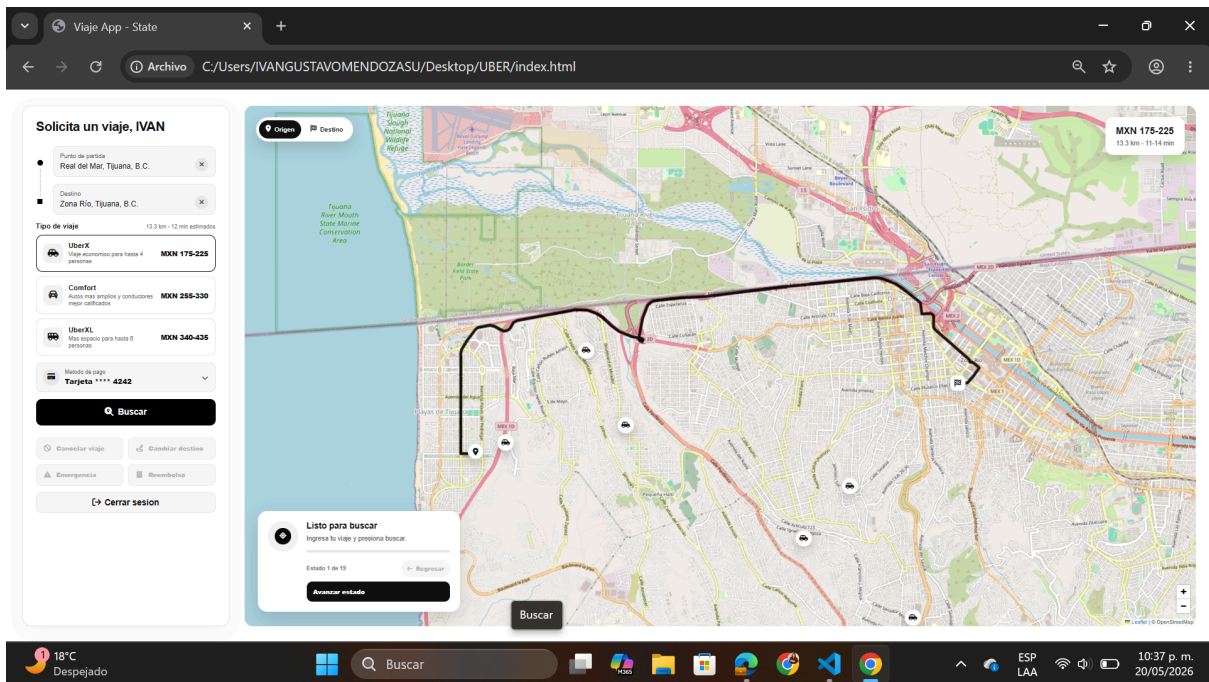
El proyecto utiliza mapas interactivos mediante Leaflet y divide toda la lógica del sistema en módulos organizados.



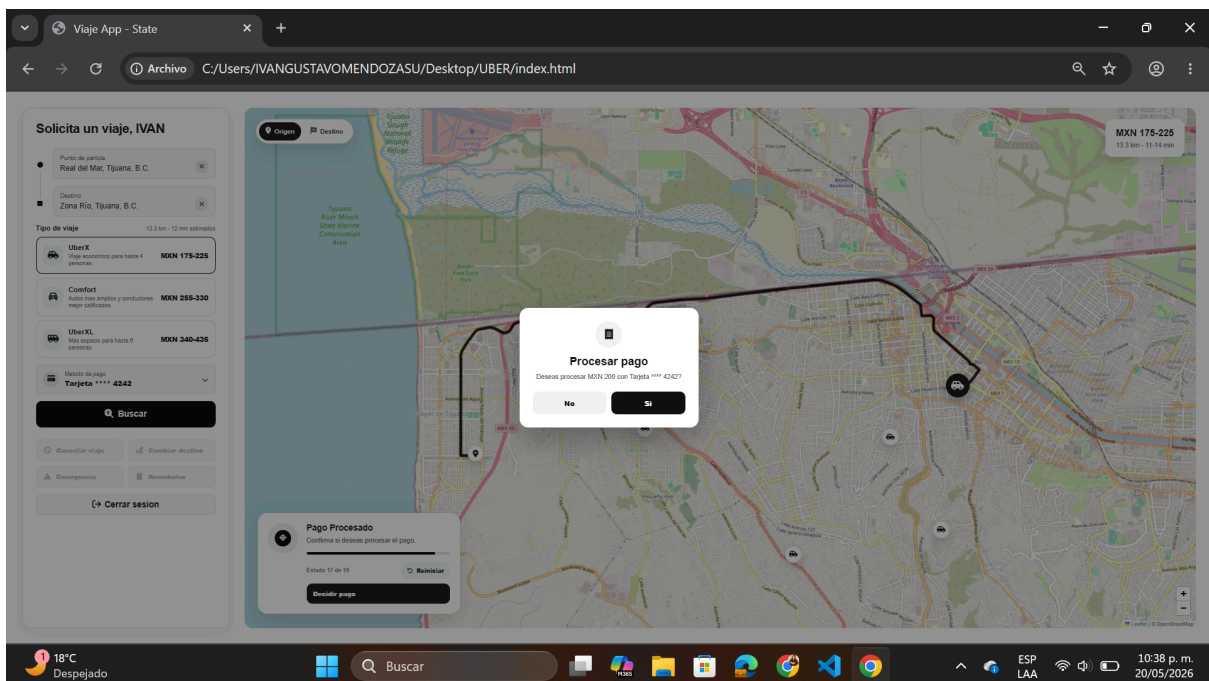
EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNM
TECNOLOGICO NACIONAL DE
MEXICO



1.2 Usuario solicita el servicio.



1.3 Finaliza servicio al usuario.



Patrones de Diseño utilizados

Patrón	¿Para qué se utilizó?	Justificación	Compatibilidad con el proyecto
State	Controlar los estados del viaje dentro de la aplicación.	Permite separar la lógica de cada etapa del viaje y evita grandes bloques de condicionales. Facilita agregar nuevos estados y mejora el mantenimiento.	Es compatible porque la aplicación funciona mediante diferentes etapas consecutivas como búsqueda de conductor, viaje en curso y finalización.
Strategy	Manejar distintos métodos de pago y cálculos de tarifa.	Permite cambiar algoritmos sin modificar la lógica principal del sistema. Reduce el acoplamiento y mejora la escalabilidad.	Encaja perfectamente porque la aplicación necesita manejar diferentes tipos de pagos y tarifas dinámicas.
Factory	Crear objetos como conductores, pagos y tipos de viaje.	Centraliza la creación de objetos y evita duplicación de código. Mejora la organización y reutilización.	Compatible debido a que el sistema genera múltiples tipos de objetos durante la ejecución.
Observer	Actualizar automáticamente la interfaz cuando cambia el estado del viaje.	Permite que diferentes componentes reaccionen automáticamente a cambios dentro del sistema.	Es útil porque la aplicación necesita reflejar cambios en tiempo real en el mapa y paneles visuales.
Command	Encapsular acciones del usuario como comandos ejecutables.	Facilita controlar y validar acciones del usuario de forma organizada.	Compatible porque el sistema depende constantemente de acciones como confirmar viajes y seleccionar opciones.
Adapter	Integrar la librería Leaflet para el manejo de mapas.	Reduce la dependencia directa con librerías externas y mejora el mantenimiento.	Compatible porque la aplicación utiliza mapas interactivos para mostrar rutas y ubicaciones.

Compatibilidad entre los patrones

Todos los patrones implementados trabajan de manera conjunta dentro del proyecto.

Relación entre patrones

- El patrón State controla el flujo del viaje.
- Observer actualiza la interfaz cuando cambia un estado.
- Strategy calcula tarifas y pagos según la selección.
- Factory crea los objetos necesarios del sistema.
- Command ejecuta acciones del usuario.
- Adapter conecta el sistema con los mapas.

Gracias a esta combinación:

- El código es más modular.
- El sistema es más fácil de mantener.
- Las funciones están desacopladas.
- Es más sencillo agregar nuevas características.



Diagrama UML

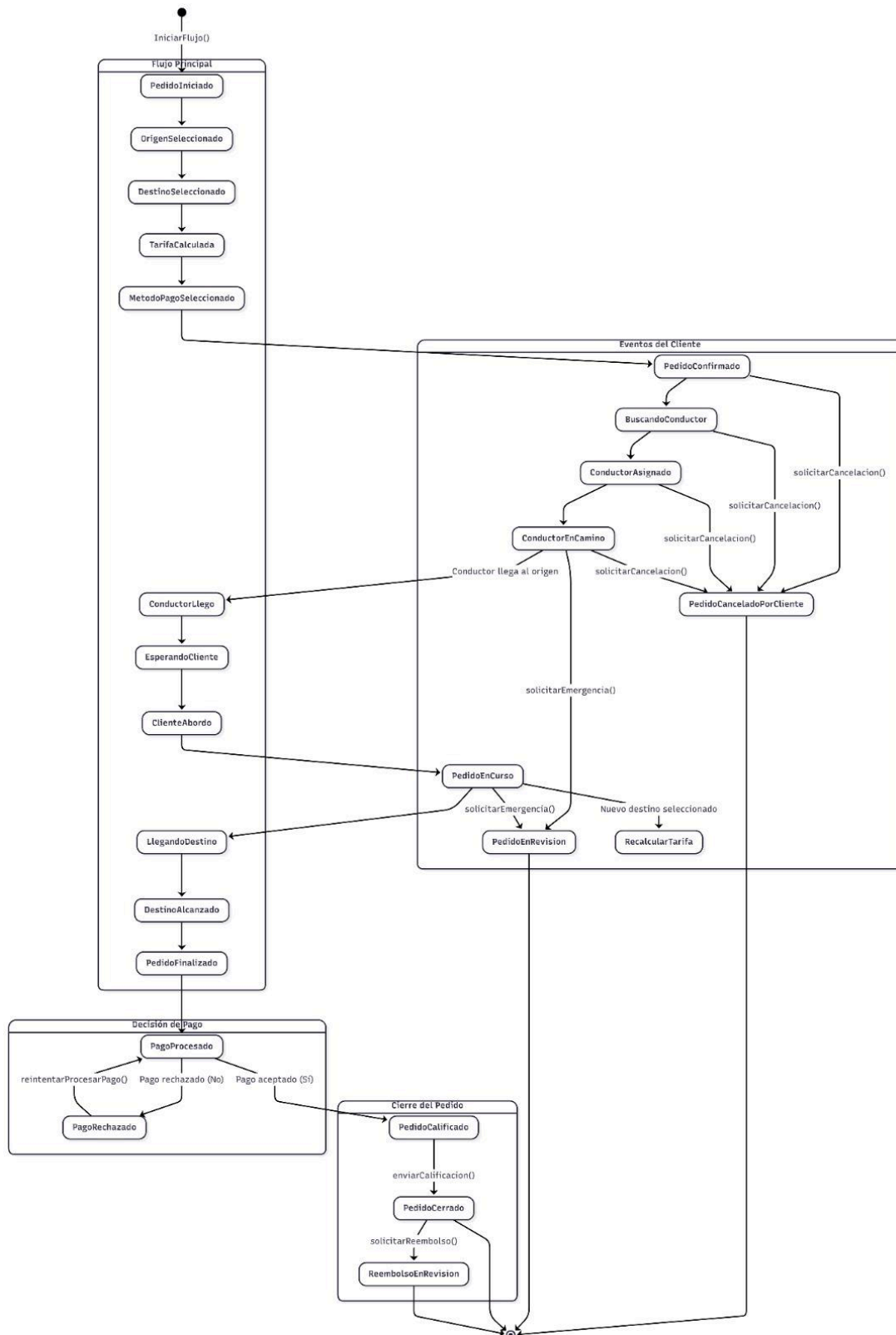
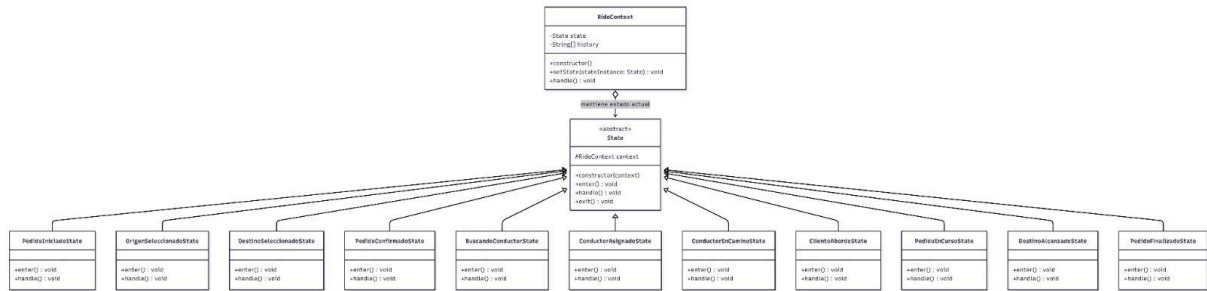


Diagrama de estados



Conclusiones

-Felix

Este proyecto permitió aplicar patrones de diseño en una simulación funcional de solicitud de viajes. La separación del código en módulos hizo que la aplicación fuera más ordenada, fácil de mantener y más clara para entender cómo trabajan patrones como State, Strategy, Factory, Observer, Command y Adapter. Además, la interfaz integra el flujo completo del viaje, desde la selección de ruta hasta el pago, mostrando cómo estos patrones pueden usarse en un caso práctico y realista.

-Ivan

Este proyecto me ayudó a entender mejor cómo funcionan los patrones de diseño y cómo se pueden aplicar en una aplicación real. Gracias a los patrones utilizados, el código quedó más organizado, más fácil de entender y más sencillo de mantener además, pude ver cómo cada patrón cumple una función importante dentro del sistema y cómo todos trabajan juntos para mejorar el funcionamiento de la aplicación. También aprendí la importancia de estructurar bien un proyecto para facilitar futuras mejoras o nuevas funciones y finalmente, considero que este proyecto fue una buena práctica para reforzar mis conocimientos de programación y comprender mejor el desarrollo de software utilizando buenas prácticas.

-Yahir

Durante este proyecto aprendí mucho sobre la manera en que se desarrolla una aplicación utilizando patrones de diseño. Al ir trabajando cada parte del sistema, entendí cómo una buena organización del código puede hacer que todo funcione mejor y sea más fácil de modificar después. También me ayudó a practicar la lógica de programación y a ver cómo se conectan distintas funciones dentro de una aplicación real. En general, fue una experiencia que me sirvió para mejorar mis conocimientos y tener una idea más clara de cómo se trabaja en proyectos de software más completos.

-Rodrigo

Este proyecto me permitió comprender de una forma más práctica cómo se implementan los patrones de diseño dentro de una aplicación real. A lo largo del desarrollo, pude identificar la importancia de mantener una buena estructura y separación de responsabilidades en el código para facilitar su mantenimiento y escalabilidad. Además, trabajar con patrones como State, Strategy, Factory, Observer, Command y Adapter me ayudó a entender cómo cada uno resuelve problemas específicos dentro del sistema. También reforcé mis conocimientos en programación y adquirí una mejor visión sobre cómo se desarrollan aplicaciones más organizadas, reutilizables y fáciles de modificar en futuros proyectos.